DASAR PEMROGRAMAN

Pengantar Dasar Pemrograman

Algoritma, Problem Solving, Pseudocode, dan Flowchart

Pertemuan 1

1. **Dr. Desta Sandya Prasvita, S.Komp, M.Kom**
2. **Ichsan Mardani, S.Kom., MSc.**
3. **Nurul Afifah Arifuddin, S.Pd., M.T**
4. **Dr. Didit Widiyanto, S.Kom., M.Si.**

Program Studi S1 Informatika | Universitas Pembangunan Nasional Veteran Jakarta

Roadmap Mata Kuliah Dasar Pemrograman

Semester 1 • 16 Pertemuan

Fase 1 — Fondasi Pemrograman

1. Pengantar Dasar Pemrograman
2. Input, Output, Variabel, dan Tipe Data
3. Operator dan Ekspresi
4. Percabangan
5. Perulangan
6. Perulangan Bersarang dan Debugging
7. Fungsi dan Rekursi
8. **Ujian Tengah Semester**

Fase 2 — Representasi dan Pengolahan Data

9. Array Satu Dimensi
10. Array Dua Dimensi
11. Karakter dan String
12. Pointer
13. Struct dan Representasi Data
14. Searching dan Sorting
15. File dan Pengujian Program
16. **Ujian Akhir Semester / Project Akhir**

Alur Pembelajaran: Problem Solving → Algoritma → Coding → Testing

Komponen Penilaian Mata Kuliah Dasar Pemrograman

Semester 1 • Sistem Evaluasi

1 Keaktifan

Partisipasi aktif dalam diskusi, tanya jawab, dan kegiatan kelas.

10%

2 Tugas Harian

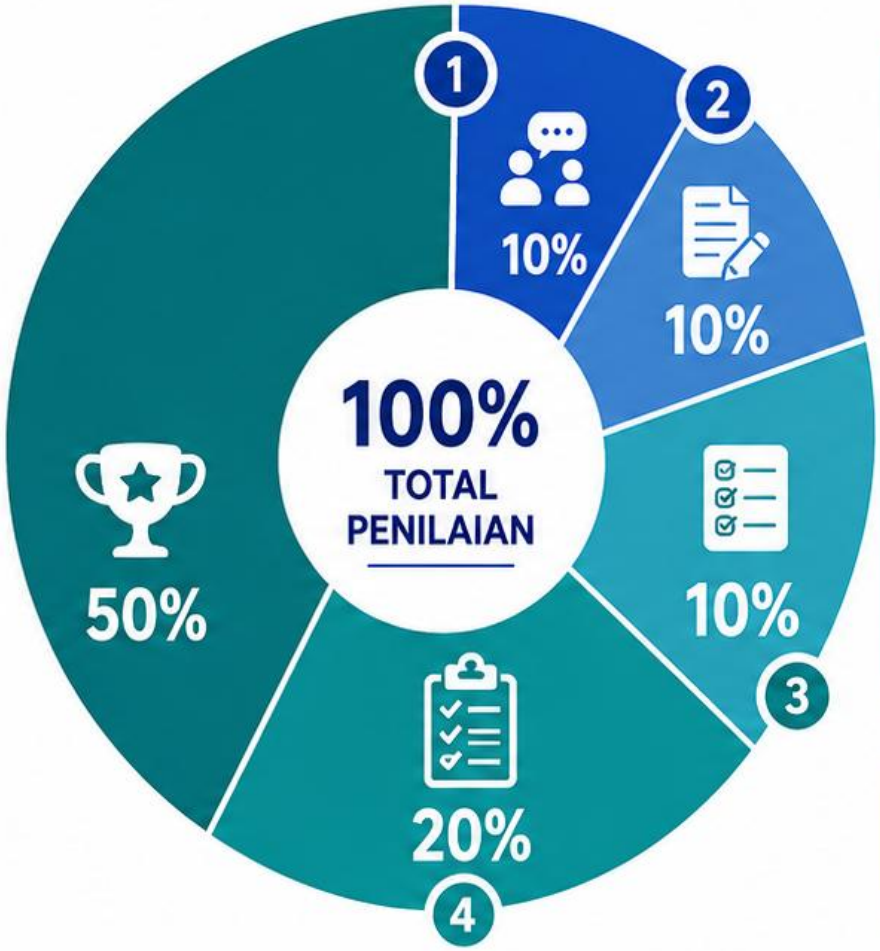
Penyelesaian tugas individu/kelompok secara rutin.

10%

3 Quiz Harian

Quiz singkat untuk mengukur pemahaman konsep secara berkala.

10%



4 UTS

Ujian Tengah Semester menguji pemahaman materi sampai pertengahan perkuliahan.

20%

5 UAS / Project Akhir

Ujian Akhir Semester atau proyek akhir terintegrasi yang mencakup penerapan konsep secara menyeluruh.

50%

Total Penilaian = 100%

Fokus evaluasi: partisipasi, latihan rutin, pemahaman konsep, dan proyek akhir terintegrasi.

Kontrak Perkuliahan Dasar Pemrograman

Komitmen Belajar Semester 1

1



Kehadiran

Hadir tepat waktu dan mengikuti perkuliahan secara aktif.

2



Partisipasi

Aktif bertanya, menjawab, dan berdiskusi di kelas.

3



Pengumpulan Tugas

Tugas dikumpulkan sesuai jadwal yang telah ditentukan.

4



Quiz/Ujian

Quiz, UTS, dan UAS dikerjakan secara jujur dan mandiri.

5



Etika Akademik

Hindari plagiarisme dan salin-tempel tanpa memahami isi tugas.

6



Penggunaan AI

AI boleh membantu belajar, tetapi mahasiswa tetap wajib memahami kode.

7



Komunikasi dengan Dosen

Gunakan bahasa yang sopan dan komunikasi yang profesional.

Tujuan utama kontrak ini adalah menciptakan suasana belajar yang disiplin, jujur, dan kolaboratif.

Gambaran Project Akhir / Target Akhir Mata Kuliah

Aplikasi Sederhana Bahasa C yang Mengintegrasikan Materi Dasar Pemrograman

1. **Input & Output**
   Menerima dan menampilkan data
2. **Percabangan**
   Mengambil keputusan dengan logika program
3. **Perulangan**
   Memproses data secara berulang
4. **Fungsi**
   Membagi program menjadi bagian modular

Project Akhir

Membangun Aplikasi Sederhana Berbasis C

5. **Array / String**
   Mengelola kumpulan data sederhana
6. **Struct**
   Merepresentasikan data terstruktur
7. **Searching & Sorting**
   Mencari dan mengurutkan data
8. **File & Testing**
   Menyimpan data dan menguji program

Output yang Diharapkan

- **Source Code**
  Program C dapat dijalankan
- **Fitur Utama**
  Menu dan fungsi berjalan dengan benar
- **Pengujian**
  Ada test case sederhana
- **Presentasi / Demo**
  Mahasiswa mampu menjelaskan program

Target akhir mata kuliah ini bukan hanya membuat program, tetapi membangun cara berpikir logis, sistematis, dan terstruktur melalui proyek sederhana.

Alur Cara Belajar Dasar Pemrograman

— Tahapan Belajar yang Sistematis —

1



Memahami Masalah

Pahami persoalan yang akan diselesaikan.

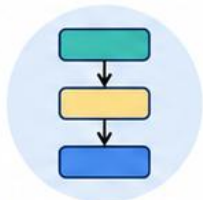
2



Analisis Input & Output

Tentukan data masukan dan hasil yang diharapkan.

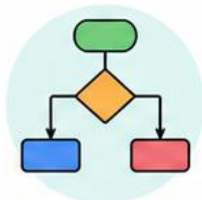
3



Susun Algoritma

Rancang langkah-langkah penyelesaian secara logis.

4



Flowchart / Pseudocode

Representasikan algoritma dalam bentuk visual atau naratif.

5



Tulis Kode C

Implementasikan algoritma ke dalam program bahasa C.

6



Compile & Run

Jalankan program dan periksa apakah kode dapat dieksekusi.

7



Testing

Uji program dengan beberapa data untuk melihat hasilnya.

8



Debugging

Temukan dan perbaiki kesalahan hingga program berjalan benar.



Belajar pemrograman bukan hanya menulis kode, tetapi juga berpikir logis, sistematis, dan terstruktur.



DISKUSI SINGKAT

Apa sebenarnya yang dilakukan seorang programmer?

BUKAN HANYA

Menulis kode

TETAPI

Memahami masalah → Merancang solusi → Menuliskan solusi

Pemrograman dan Problem Solving

Pemrograman adalah proses penyelesaian masalah menggunakan komputer.



Poin Penting: coding hanyalah salah satu bagian dari proses pemrograman — bukan keseluruhannya.

Contoh Masalah Sederhana

STUDI KASUS PEMBUKA

Menghitung rata-rata tiga nilai mahasiswa

Sebelum diberikan solusi, coba pikirkan dahulu:

? Data apa yang dibutuhkan?

? Proses apa yang dilakukan?

? Apa hasil yang diharapkan?

Input–Process–Output

Menggunakan contoh sebelumnya: menghitung rata-rata tiga nilai mahasiswa

INPUT

```
nilai1
nilai2
nilai3
```

PROCESS

```
rata = (nilai1 + nilai2
        + nilai3) / 3
```

OUTPUT

```
nilai rata-rata
```

Apa Itu Algoritma?

Algoritma adalah urutan langkah yang **logis dan sistematis** untuk menyelesaikan suatu masalah.

Contoh sederhana: Algoritma Membuat Teh

1. Siapkan gelas
2. Masukkan teh
3. Tuangkan air panas
4. Tambahkan gula
5. Aduk
6. Sajikan



Algoritma dalam Pemrograman

Contoh: Menghitung Luas Persegi Panjang

1. Masukkan panjang
2. Masukkan lebar
3. Hitung luas
4. Tampilkan luas

Perhatikan: komputer membutuhkan instruksi yang jelas dan berurutan — tidak dapat menerka maksud yang ambigu.

Karakteristik Algoritma

Algoritma yang baik memiliki ciri-ciri berikut:

Langkah Jelas

Setiap instruksi mudah dipahami, tidak ambigu.

Input (jika diperlukan)

Dapat menerima data masukan dari luar.

Menghasilkan Output

Ada hasil yang jelas di akhir proses.

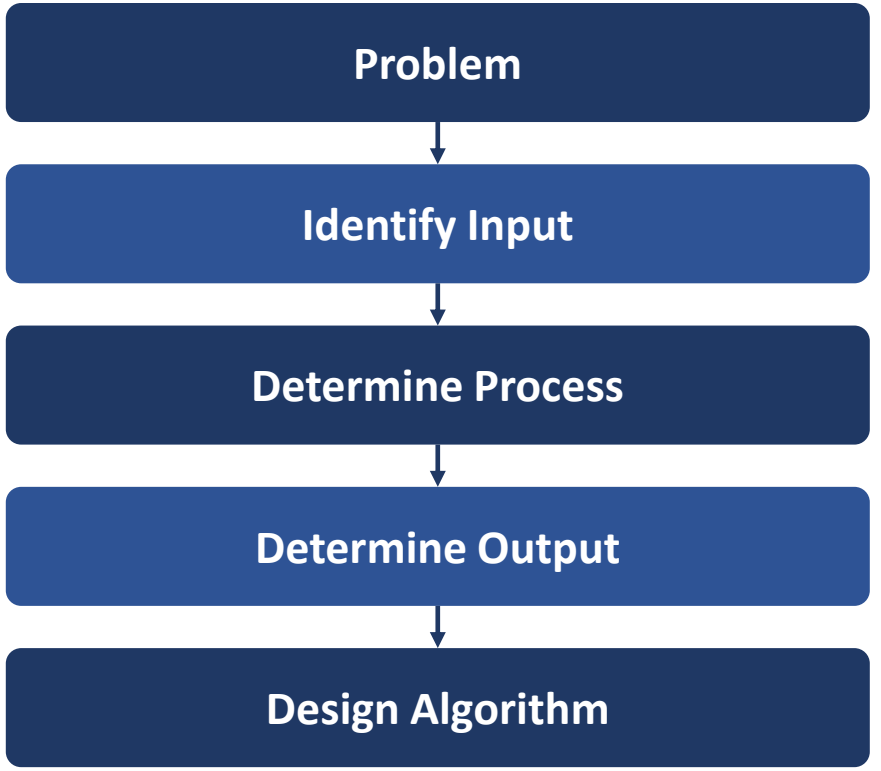
Dapat Dilaksanakan

Setiap langkah benar-benar bisa dikerjakan.

Memiliki Kondisi Selesai

Algoritma berhenti pada titik tertentu, tidak berjalan selamanya.

Dari Masalah Menjadi Solusi



Contoh: `Total belanja = harga × jumlah`

Bagaimana Menuliskan Algoritma?

Dua representasi umum digunakan untuk menuliskan algoritma:

1. Pseudocode

Berbasis teks.
Menyerupai kode program tanpa terikat sintaks bahasa tertentu.

2. Flowchart

Berbasis visual.
Menggunakan simbol-simbol standar yang dihubungkan garis alur.

Pesan Utama: keduanya membantu kita merancang logika sebelum menulis program.

Pseudocode

Pseudocode adalah cara menuliskan algoritma menggunakan bahasa sederhana tanpa terikat sintaks bahasa pemrograman tertentu.

Contoh: Menghitung Luas Persegi Panjang

```
START
    INPUT panjang
    INPUT lebar
    luas ← panjang × lebar
    OUTPUT luas
END
```

Mengapa Menggunakan Pseudocode?

- Fokus pada logika
- Mudah dibaca
- Mudah diperbaiki
- Tidak tergantung bahasa pemrograman
- Membantu sebelum coding

Think first,
code later.

Flowchart

Flowchart adalah suatu bagan atau diagram dengan simbol-simbol tertentu yang menggambarkan urutan proses secara mendetail dan hubungan antara suatu proses (instruksi) dengan proses lainnya dalam suatu program.



Start / End



Input / Output



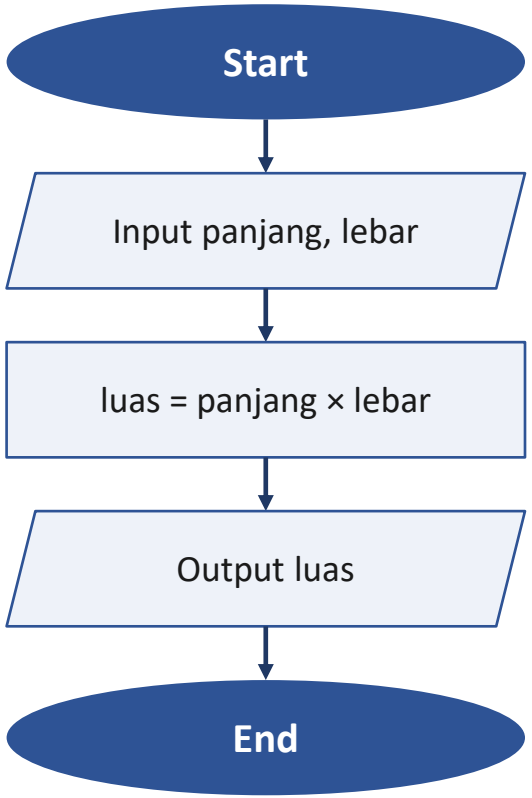
Process



Flow

Contoh Flowchart Sederhana

Kasus: Menghitung Luas Persegi Panjang

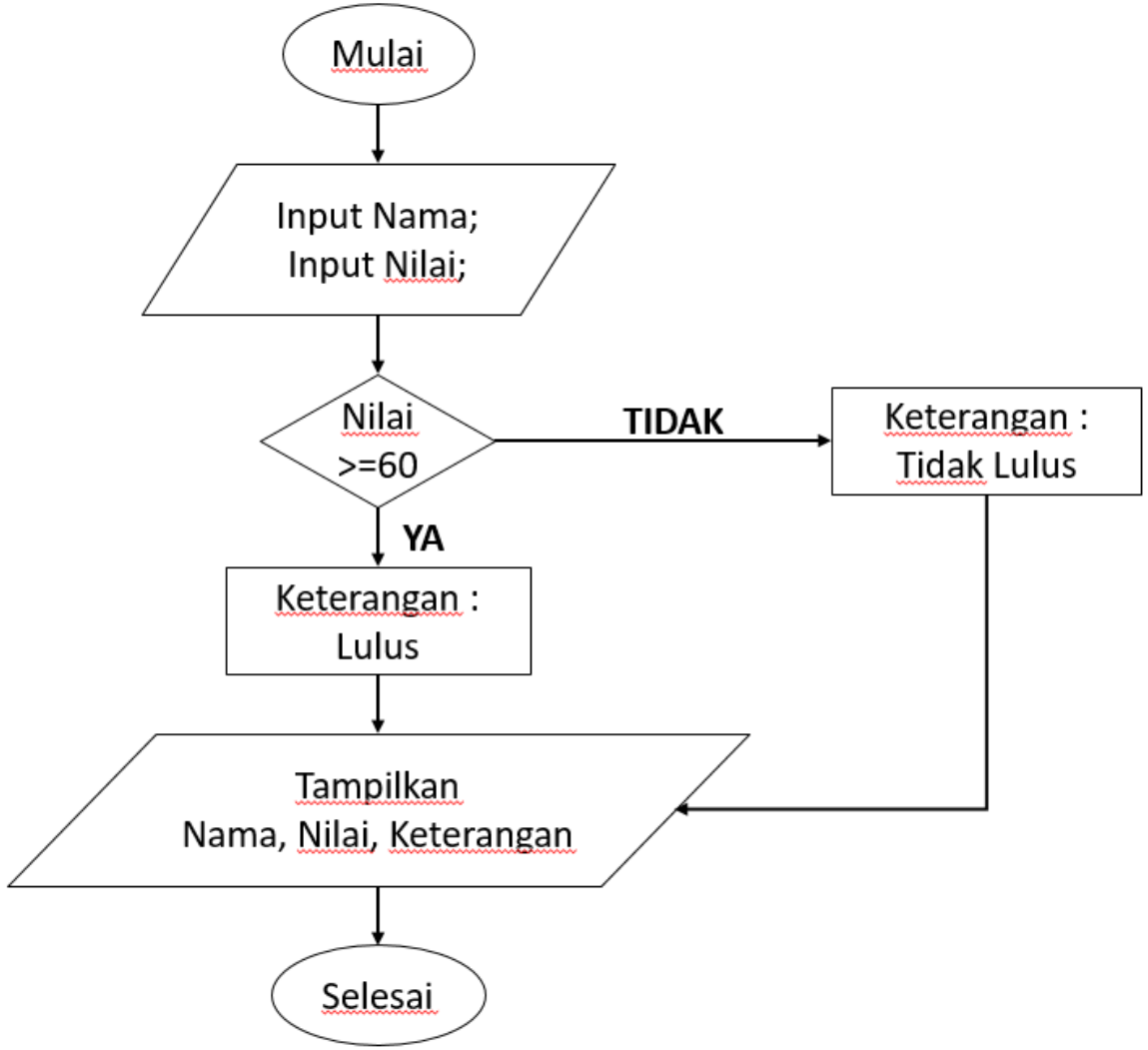


Catatan

Flowchart ini sekaligus mengenalkan alur sekuensial: setiap simbol dijalankan tepat satu kali, dari atas ke bawah, tanpa lompatan atau pengulangan.

Contoh Algoritma Flowchart

SIMBOL	NAMA	FUNGSI
	TERMINATOR	Permulaan/akhir program
	GARIS ALIR (FLOW LINE)	Arah aliran program
	PREPARATION	Proses inisialisasi/pemberian harga awal
	PROSES	Proses perhitungan/proses pengolahan data
	INPUT/OUTPUT DATA	Proses input/output data, parameter, informasi
	DECISION	Perbandingan pernyataan, penyeleksian data yang memberikan pilihan untuk langkah selanjutnya
	ON PAGE CONNECTOR	Penghubung bagian-bagian flowchart yang berada pada satu halaman
	OFF PAGE CONNECTOR	Penghubung bagian-bagian flowchart yang berada pada halaman berbeda



Pseudocode vs Flowchart

Pseudocode	Flowchart
Berbasis teks	Berbasis visual
Cepat ditulis	Alur mudah dilihat
Mudah diubah	Cocok untuk komunikasi
Mendekati program	Menggunakan simbol

Kesimpulan: keduanya merepresentasikan algoritma yang sama, hanya bentuknya berbeda.

Dari Algoritma Menjadi Program



Algoritma

Solusi logis terhadap masalah.

Program

Implementasi algoritma menggunakan bahasa pemrograman.

Bahasa Pemrograman

- Kumpulan instruksi yang digunakan untuk mengatur komputer agar melakukan suatu tindakan tertentu dinamakan program.
- Instruksi yang digunakan dalam menulis program disebut bahasa pemrograman.
- Orang yang membuat program komputer dinamakan pemrogram atau programmer.
- Kegiatan merancang dan menulis program komputer disebut pemrograman atau programming.



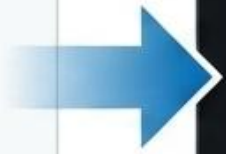
Untuk mata kuliah ini, bahasa implementasi: C

Tetapi prinsip yang dipelajari: problem solving dan algoritma dapat diterapkan pada berbagai bahasa pemrograman.

Algoritma ke Bahasa C

Pseudocode

```
INPUT panjang
INPUT lebar
luas ← panjang × lebar
OUTPUT luas
```



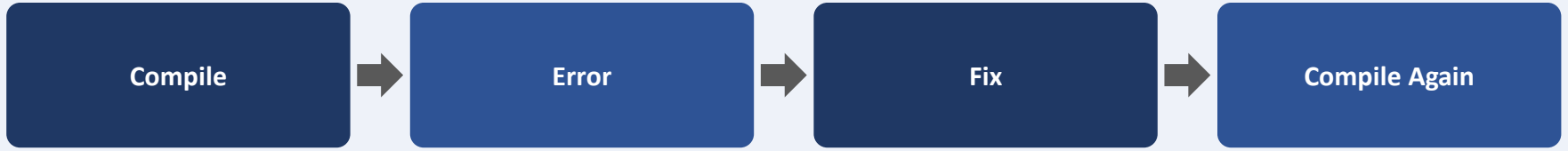
Kode C

```c
scanf("%d", &panjang);
scanf("%d", &lebar);
luas = panjang * lebar;
printf("%d", luas);
```

Bagaimana Program C Dijalankan?



Jika terjadi error:



Compiler dan Syntax Error

Compiler

Menerjemahkan source code menjadi program yang dapat dijalankan komputer.

Contoh Syntax Error

```
printf("Hello")          // tanda ; hilang
```

Pesan Utama: error message membantu programmer menemukan masalah dalam program.

Program Berjalan ≠ Program Benar

Input

```
panjang = 10
lebar = 5
```

Expected Result

```
50
```

Actual Result

```
15
```

Program berhasil dijalankan, tetapi logikanya salah.

"Running successfully does not always mean correct."

Ini menjadi pengantar konsep testing.

Tracing

Tracing adalah menelusuri proses algoritma atau program langkah demi langkah.

Kode

```
a ← 10
b ← 5
c ← a + b
a ← c × 2
```

Langkah	a	b	c
a ← 10	10	-	-
b ← 5	10	5	-
c ← a+b	10	5	15
a ← c×2	30	5	15

Nilai akhir: a = 30, b = 5, c = 15

STUDI KASUS

Menghitung Rata-rata Tiga Nilai

Mahasiswa menentukan:

1. Input
2. Process
3. Output
4. Pseudocode
5. Flowchart

Data

```
80, 75, 85
```

Hasil

```
Rata-rata = 80
```

Kesalahan Umum Pemula

1. Langsung Coding

Tanpa memahami masalah terlebih dahulu.

2. Algoritma = Program?

Menganggap keduanya sama, padahal berbeda.

3. Menghafal Kode

Tanpa memahami proses di baliknya.

4. Asumsi Selalu Benar

Menganggap program yang berjalan pasti benar, padahal tetap harus diuji.

LATIHAN KELAS

Kasus: Total Pembelian

Input

- harga barang
- jumlah barang

Tugas mahasiswa:

1. Tentukan IPO
2. Buat pseudocode
3. Buat flowchart
4. Gunakan contoh data dan lakukan tracing

CHALLENGE QUESTION

Apakah dua algoritma berbeda dapat menghasilkan hasil yang sama?

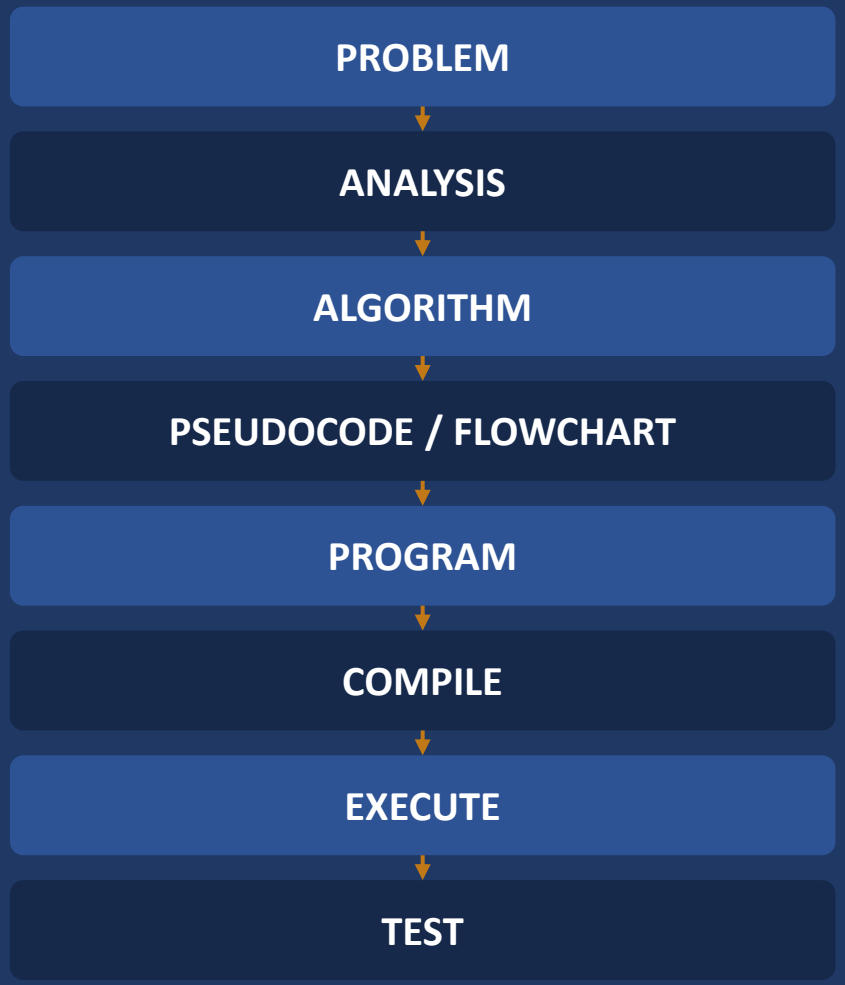
Ya.

Karena satu masalah dapat memiliki lebih dari satu solusi. Ini menjadi bibit pembahasan efisiensi algoritma pada pertemuan selanjutnya.

Rangkuman

Mahasiswa hari ini mempelajari:

- **Problem Solving**
- **Algorithm**
- **Input–Process–Output**
- **Pseudocode**
- **Flowchart**
- **Program**
- **Programming Language**
- **Compiler**
- **Testing**
- **Tracing**



“Programming starts with thinking, not coding.”